1 The plugin runs the same engine we already run (read from v2.1.220, 260731)

What is inside the VS Code plugin: its three layers, and which one really talks to Claude.

```
the plugin is a HOST, not an agent
  +--------------+   packs in  +--------------+   starts  +--------------+
  |  webview UI | ------------> |  Agent SDK | ----------> |  claude CLI |
  +--------------+               +--------------+             +--------------+
   no new agent    no new message format
  [x] only a shell around an engine we run
```

This part settles that the VS Code plugin builds no engine of its own, so there is nothing hidden for us to copy. The plugin does not build an agent of its own. It does not build its own message format either. It packs in the TypeScript Agent SDK and drives it, and `extension.js` gives that away in its own option names. - `pathToClaudeCodeExecutable` · `canUseTool` · `includePartialMessages` · `settingSources` · `permissionPromptToolName` · `forkSession` · `sessionMirror` Our Python `claude_agent_sdk` offers those same option names, because it is the same SDK in two languages.

```
  webview/index.js    5MB    the chat UI: bubbles, diffs, tool cards. web chat, never a termi
  extension.js      2.5MB    the host: packed-in Agent SDK + VS Code glue + the IDE bridge se
  resources/native-binary/claude   245MB   a packed copy of the ORDINARY CLI, started as a su
  ~/.claude/ide/<pid>.lock          the bridge handshake: the PLUGIN listens on a WebSocket
                                    and the CLI dials OUT to it (transport: "ws" + authToken)
```

That 245MB file is not a special build, and it is not a second product (JL asked, 260731). It is the Claude Code CLI compiled into one Mach-O program, with the Node runtime and every dependency baked in, and that is the whole reason it is so big. Checked on this machine: `shasum -a256` matches `~/.local/share/claude/versions/2.1.220` byte for byte, and that is exactly what `claude` on the PATH points to. The plugin carries a copy so it still works for someone who never installed the CLI. That is all the copy buys, and we need no copy, because serve.py already starts the one on the PATH. The start line is short and always the same, and then one flag per option that was set.

```
claude --output-format stream-json --verbose --input-format stream-json

then one flag per set option:
  --model  --effort  --thinking adaptive|disabled  --max-turns  --max-budget-usd
  --setting-sources=...  --allowedTools  --disallowedTools  --permission-mode
  --mcp-config  --resume=<sid>  --session-id=<sid>  --fork-session
  --include-partial-messages
```

Everything after the start line is a plain translation, the same table our own options would produce. This page owns the chat box and nothing else. That means the three permission tiers, streaming, how it draws, cost, how it handles sessions, and every part of it a reader touches. The rules about

who may hold a session belong to QD1. The real terminal is QD3, and its form on a small screen is QD4. The shell that gives the chat box its own pane is QD5. Whether a reader can TRUST what the chat box shows is tested by QF4, along BINDING · TURN · CONTINUITY · HANDOVER · INTERRUPTION. The question "were these fixes ever clicked in a browser" belongs there too. How well this page is WRITTEN belongs to QB4's grammar and QF1's checker. It is written down here on 260801 only because JL asked it on this page (" follow guideline … Q "), and the answer is that this page does not own it.

2 One pipe carries the answers and the permission asks

The four kinds of message: what travels each way between us and the CLI, on one pipe.

```
ONE pipe, four kinds of newline JSON
   assistant + deltas      -->  [x] the half the chat box already draws
   control_request         --  the CLI ASKING: a permission prompt
   control_response        -->  our allow/deny, matched by request_id
  [!] control_cancel_request  --  plus  keep_alive .  transcript_mirror
   setting can_use_tool is what switches the control half ON
```

This part settles that permission asks travel on the same pipe as the answers, and that one setting turns that half on. One pipe carries four kinds of traffic, and all of them are JSON, one object per line. The answer text and its partial pieces arrive as ordinary messages, and that is the half our chat box already draws. The other three are the half we do not use yet. - `control_request` · the CLI asking, and this is how a permission prompt arrives - `control_response` · our answer, matched by `request_id` - `control_cancel_request` `keep_alive` and `transcript_mirror` messages ride the same channel. Setting `canUseTool` is what turns that channel on. The SDK then adds `--permission-prompt-tool stdio`, and every allow or deny travels as a control message instead of on a side channel.

3 We started a new claude for every message, and that was the whole cost

One process or many: how the plugin serves a conversation, and how serve.py used to.

```
plugin      > ONE process, MANY turns    instant follow-up
  turn1 -+
  turn2 -+-->  one live claude -->
  turn3 -+

serve.py, before M1 > one process PER TURN   8.1s first token .  ~$0.9
  turn1 -->  boot  ~150 skills -->  -->  dropped
  turn2 -->  boot  ~150 skills -->  -->  dropped
                    the WHOLE gap: we drop the client every POST
```

This part settles that the slow, costly first token came from dropping the client after every message, and not from anything about the SDK. The plugin's read loop runs ONCE for the life of a session, and it pushes each new user turn into the live process with `inputStream.enqueue(...)`.

That is what `--input-format stream-json` buys: stdin is a STREAM of turns, not a single prompt, so one process serves the whole conversation. Our `ClaudeSDKClient` can do exactly the same. Its own docstring even names our use case ("Building chat interfaces or conversational UIs", "Multi-turn conversations with context"). serve.py threw that away. `chat()` opened `async with ClaudeSDKClient(...)` inside a per-POST `anyio.run(run)`, so every message connected, ran one turn, and disconnected. That single line was the whole "not that good". The 8.1s first token and the near-0.9$full-tiermessagewerebothth ecostofconnectingagainandloadingthe$ 150 − $skilllistagain, oncepermessage.Theblockerwasjustasspecific, anditwasnotthemessageformat.TheSDKforbids$ $livedevent-loopthreadthatownseveryliveclient, withqueuesinandout.TheHTTPhandlerstopsowningtheloopand$

4 What it takes to match the plugin, in four steps

The four steps: what M1 unlocks, and what falls out of it for free.

```
M1 the session host  -- unlocks everything below --+
  one daemon thread . one loop . SESSIONS[question]  |
                                                     +->  M2 streaming verbs
                                                     |      interrupt . set_model
                                                     |      set_permission_mode
                                                     |      get_context_usage
                                                     +->  M3 rewind_files()
                                                     +->  M4 @-mentions . plan mode
   M4 is OURS to build; M2 and M3 are option flips once M1 lands
```

This part settles that one build, M1, unlocks the rest: M2 and M3 become option flips, and only M4 is real work of our own. - M1 · the session host A daemon thread runs one asyncio loop for the life of the process, and `SESSIONS[question] -> {client, inbox, outbox}` holds the clients. `chat()` stops calling `anyio.run`. It hands the message to the loop, then drains that session's outbox into the NDJSON stream we already have, so the browser side does not change at all. Idle reaping and the `QD1` HOLD keep their current rules. - M2 · the streaming-mode verbs, free once M1 lands Four `ClaudeSDKClient` methods work ONLY in streaming mode, and none of them can be reached today. `interrupt()` replaces our flag that waits for the next message boundary. `set_model()` and `set_permission_mode()` switch mid-conversation with no reboot. `get_context_usage()` gives the chat box a real context meter. - M3 · rewind, the feature we had parked `rewind_files(user_message_id)` is save points and rewind, listed on this page as "parked, it fights the LAW". It stops fighting. The amended `QD1` Law already allows many sessions per question, and rewind works inside one session instead of forking a second history. It needs `enable_file_checkpointing=True` plus `replay-user-messages`, and both are option flips. - M4 · the two things in the interface that are ours to build `@`-file mentions need a picker over the repo and a path put into the message. Plan mode is `--permission-mode plan`, which M2's `set_permission_mode()` already reaches.

5 Moving to JavaScript would buy us nothing (JL asked 260731)

Three things called "the JS version": which of them we could use, and what each one would gain.

```
"the JS version" is THREE things, with three answers
```

```
 extension.js    require("vscode") throughout . cannot even LOAD outside the IDE
 the npm SDK     the SAME SDK in another language . 0 capability gained
[x] stay Python     47 options already cover every flag the plugin emits
 the slowness is the DROPPED CLIENT, not the language
```

This part settles that we stay in Python, because the slowness came from dropping the client and not from the language. Three different things get called "the JS version", and they have three different answers. - `extension.js` itself: NO, and not for taste reasons. It is a VS Code plugin-host module that calls `require("vscode")` all through, exports no public API, and ships as one 2.5MB minified bundle. Outside VS Code the `vscode` module does not exist, so the file cannot even load. - The SDK it packs in: yes, we could reuse it, and it is on npm as `@anthropic-ai/claude-agent-sdk`. But it is the SAME SDK as our Python one, so taking it is a change of language, not a change of what we can do. - Parity, checked rather than assumed: `ClaudeAgentOptions` in Python carries 47 options, and they cover every flag the plugin's arg builder emits. There is nothing we would gain by switching. - the 47 include `thinking`, `effort`, `task_budget`, `can_use_tool`, `include_partial_messages`, `fork_session`, `skills`, `sandbox`, `plugins` and `enable_file_checkpointing` - `extra_args` is the escape hatch for any flag it has not mapped So the advice is to stay in Python, and the strongest evidence is what else serve.py is (JL asked 260731, "what does serve.py do besides the claude code?"). Measured on 260731 it was 2938 lines across 20 HTTP routes plus the terminal WebSocket, with chat one job out of seven. The big file has since been split. Today `cli/serve.py` is a 496-line HTTP router. The seven jobs live as `live/` modules: `chat.py`, `term.py`, `activity.py`, `write.py`, `xcal.py`, `structure.py`, `shell.py`, `home.py`. Counting lines of method body per area, as measured then:

```
443  excalidraw     proxy the app . per-page frames . save scenes . hydrate and
                    stash embedded images . attach a drawing to a page   (QB4b, QD5)
414  chat           the Claude Code bridge                                (QD2)
390  activity       a SQLite focus-time database: spans per board, group,
                    page and actor, day-part aggregation, cross-board stats (QD6)
334  write-back     every comment, lane, edit, discussion and resolve landed as a
                    typed line under its exact anchor sentence            (QB8)
322  terminal       the PTY serve.py now owns, its WS terminus, ring buffer,
                    resize and reaping                                    (QD3)
291  http           routing, headers, target resolution, and calling build.py
                    to regenerate board.html after every write
 48  image paste . HOLD locking . structure edits (add Q, add group, archive)
~670 module level   the four rules texts, prime_context, tool_brief, _slugify,
                    structure_op, the PTY helpers
```

Going JS therefore means porting an Excalidraw round trip, a SQLite database, a sentence-marking engine, and a PTY, and none of them touch Claude Code. All of that work, to change the language of the one part that already matches the plugin completely. `build.py` `check.py` `xcal.py` `lanes.py` `skillpage.py` are Python as well. The point that settles it: the slowness is not Python against JS. It is that we drop the client on every POST. So changing the language alone would fix nothing, and holding the client fixes everything.

6 The board becomes the plugin, layer by layer (JL 260731)

Plugin against board: which piece of the VS Code plugin each piece of this board already is.

```
 VS Code plugin                          this board                          state
---------------------------------     ---------------------------------   -------
 webview/index.js   the chat UI  ->   board.html's chat box                  [x]
 extension.js       the host     ->   serve.py                               [x]
 vscode.postMessage the wire     ->   POST /_board/chat + NDJSON             [x]
 packed-in SDK      the engine   ->   claude_agent_sdk  (same SDK)           [x]
 the claude binary               ->   the same binary, from PATH             [x]
 the workbench      one long-    ->   QD5's shell: the chat pane is its      [x]
                      lived UI         own document, and it docks                 260802
 the RETAINED       a hidden     ->   the ring: a turn survives its
   webview            panel keeps      reader, and a returning reader              A9.1
                      its state        re-attaches at a cursor
 IDE bridge (WebSocket)          ->    nothing yet, and this is the interesting one
```

This part settles that every layer of the plugin already has its match on this board, and it names the one layer that does not. The goal is not "make the chat box more like the plugin". It is that the board becomes the plugin, and the map is one to one at every layer. The IDE bridge exists so a session can reach things inside the EDITOR: a diff in a real tab, the current selection, the language server's warnings. On this board the editor IS the board page, so the same thing already half exists under different names. The sentence address is the selection, `QB8`'s `>` lines are the notes, and `check.py`'s output is the warnings. That is the one place where copying means translating instead, and it is where the board can end up better than the plugin rather than only equal to it.

7 The one piece we cannot copy, and why it does not hurt

The IDE bridge: what it reaches, and where we choose to be different.

```
the IDE bridge is the ONE piece we cannot copy
  it exists to reach things in the EDITOR:  a diff in a real tab
                                            the current selection
                                            the language server's warnings
  [x] EXACT at the engine + message-format layer
   DIFFERENT on purpose in what the reader sees   the line we hold
```

This part settles that we match the plugin at the engine and the message format, and that we differ on purpose in what the reader sees. The IDE bridge is the one piece we cannot copy. It exists to put things inside the EDITOR: a diff in a real editor tab, the current text selection, the language server's warnings. The chat box answers the same need in its own view, and it already ships the important half, the diff preview at the permission check. So "exactly the same" is exact at the engine and at the message format, and different on purpose in what the reader sees. That is the line this page already holds.

8 Four things a terminal has for free, and the chat box did not

The four things: what nobody gave the chat box, and how many defects each one caused.

```
FOUR things nobody gave the chat box        22 defects . [x]16 2 4
   IT SITS OVER THE PAGE         it is fixed OVER a page it does not control
```

```
   IT IS REBUILT PER PAGE        build.py puts it into every page, so it dies with each
   THE REPLY IS SAVED ONCE       at `done`, the last thing the stream ever sends
   THE CONTROLS WERE MISSING     a terminal has them for free; a drawn one does not
   not four complaints about looks: state a reader can see has to be PUT somewhere,
     and three of these four rows are one mechanism away from each other (S9)
```

This part settles that the 22 defects are four missing pieces of behaviour, not four complaints about how it looks. JL asked twice on 260801 whether the day's problems should become a part named for what the reader sees, something like Mobile Usage or UI Experience. The first answer here was no, because filing them under UI would file architecture under taste. That answer was half right. The grouping by OWNER stands, because every defect arrived as a feeling about the interface and turned out to be a fact about where state lives. The NAME was a picture that told a cold reader nothing. The subject is the interface, and the Opening already says why it needs one. A terminal carries its own behaviour, a chat box we draw has to be given all of it, and this part is the list of what nobody gave it. The fourth row is new, and it is what the rename makes room for. Rows one to three are state a reader can lose, row four is a control a reader never had, and both are things a drawn interface has to be handed. Scope, corrected on JL's ask 260801 ("focus on the SDK GUI version… the problems for QD2 only"): everything below is the chat box's own behaviour. Two classes were briefly filed here and have gone back to their owners. How well this page is WRITTEN is `QB4`'s grammar and `QF1`'s checker. Whether the chat box's fixes were ever clicked in a browser is `QF4`, which already exists as "Driving the talk layer: the SDK chat version and the TUI chat version". Every row below is a thing JL hit. They were read off both session transcripts rather than from memory (`ccda0c28-ef7e-47e0-a7e1-c13abc4f4cea` + `8c9903ba-dadb-4f00-bdd1-823986cac937`, 98 user messages, 260723 14:36 → 260801 20:43).

```
IT SITS OVER THE PAGE                                          4 . [x]3 1
  [x] a wheel over the chat box scrolled the PAGE behind it
  [x] opening landed at the oldest message, not the newest
  [x] scrolling up during a live turn snapped the reader back down
   below 820px it overlays instead of docking;  Page shipped; R3  `QD5`'s
     shell, open only for a page opened alone
     + the phone is the usage mode, not the edge case (JL 20:16)

IT IS REBUILT ON EVERY PAGE                                    7 . [x]5 1 1
  [x] close  reopen rebuilt every bubble; the flash read as janky
  [x] reopening mid-turn did nothing: the guard returned above `.on`
  [x] a reload rebuilt the chat box while the terminal held the session
   a replayed session does not paint like a live one
     + markdown . tool cards . timestamps landed; permission diffs +  thinking owed
  [x] starting a new session leaves the page unchanged, with no switch
     + closed by  Sessions +  New session (A8.8, A8.10)
   history has no selectable turn boundary, so two turns cannot be compared
  [x] the chat box is built into every page at all   answered by `QD5`'s shell (S9 R2)

THE REPLY IS SAVED AT EXACTLY ONE MOMENT                       6 . [x]6
  [x] the reply appeared only after sending the NEXT message
  [x] the chat box never re-asked the server: syncFromServer had one caller
  [x] a cut-short turn dropped the text that had already arrived
```

```
  [x] a group's history was written under `<id>` and read under `G:<id>`
  [x] an in-flight turn dies with the HTTP response that carries it  S9 R1, closed
    + navigate away . switch a setting . let it run long
  [x] a ten-minute turn hits the HTTP timeout                        S9 R1, closed

THE CONTROLS WERE NEVER GIVEN                                      5 . [x]2 3
  [x]  Sessions, as a composer tab rather than a fold
  [x] the context-usage meter: ctx 38% (62k/200k)
   typing `@` does not pull a repo file into the message
   no plan-mode toggle for a read-only planning turn
   two named sessions on one page cannot be live at once
     + blocked on a `QD1` ruling: HOLD keys on the SCOPE
```

Read down the column and four remain of the original eight. The rest were closed by §9's R1 and by `QD5`'s shell. One is blocked by `QD1`, one is comparing two turns, and only two are real work of their own: `@`-mentions and plan mode, the two the page has always called chrome.

The chat box is `position:fixed` over a page it does not control, so anything it declines to handle, the page handles instead. A wheel it cannot use scrolls the page behind it. `overscroll-behavior` cannot fix that half on its own, because that property governs a scroller that has REACHED its edge, not one that never overflowed. Below 820px the chat box stops docking and covers the page outright, so "go back to the page" stops existing as an action and only "close the chat" is left. The VS Code panel docks at every width, and that one difference is what makes it read as part of the editor instead of a thing sitting on top of it. Read from the transcript 260801, and correcting how this was filed: below 820px is not an edge case to rule on later. It is the machine JL was holding when he hit the last five defects of the day (" ", 20:16). The phone is a way people use it, so docking at every width is a requirement and not a fork.

`build.py` puts the chat into every self-contained board page, so the view is made again on each one and dies with it. That is the shared root of three bugs that look separate. The chat box opened at the top because the replay ran while it was still `display:none`. A reload rebuilt it as the chat box while the terminal held the session. Navigating away killed the turn. It is also why a fix does not reach the reader until the page is reloaded, and that is the same coupling seen from the maintenance side. The VS Code split is the target, and half of it is already ours. The plugin host is a separate process that never touches the DOM, and M1's SessionHost is exactly that.

The transcript is saved when a turn reaches `done`, and `done` is the last thing the stream ever sends, so it is exactly what a cut connection loses. A turn that ends any other way leaves the question saved and the reply gone. That is why an answer seemed to arrive only after the next message was sent. The same weakness shows up in the small. The send path wrote the log under a bare id, while the load path read a group under `G:<id>`. So a group's history was written to one key and looked for under another. Both carry one lesson: a record with a single writing moment has a single moment in which to be lost. Corrected 260801 from the transcript: this page dates the defect to 260801 and to having been found "by USING it", and it is a day older than that. On 260731 between 14:34 and 15:45 the same four probes were sent fourteen times. Seven of them were the identical "Narrate one short sentence before each step", and three were the identical "Reply with exactly: HELLO". No complaint was attached to any of them. The re-sends WERE the symptom and nobody read them as one. That is the strongest argument on this page for the repeatable check: a defect that only shows up as a person quietly retrying is invisible to every method except driving it.

The first three rows are state a reader can LOSE. This one is a control a reader never HAD,

and it is the row that most directly answers why a terminal felt better. A terminal hands you scrollback, a session list and a status line for free, because the emulator has always had them. Every one of those is a thing the chat box must be handed one at a time. The evidence is that each one arrived only when JL asked for it by name. The session picker came on 260801 17:35 (" button Sessions"), the context meter at 17:46 (" context usage "), and each took an afternoon. What is still missing is `@`-mentions and plan mode. They stay last on purpose, because they sit on top of a chat box whose state problems are the thing actually felt.

9 One build closes most of them, and the terminal already had it

Terminal against chat: the ring the terminal keeps, and the socket chat wrote to instead.

```
QD3 terminal . already right                QD2 chat . the gap (closed by R1)
  PTY master fd                                 claude_agent_sdk
       | one reader thread  term.py:195               | one HTTP handler

   ring bytearray  256KB                       self.wfile.write()  chat.py:637
     RING_CAP  base.py:65                           the response socket ITSELF
       +-->  client A                              no ring
       +-->  client B                              no client list
       +-->  client C                              no replay
   reconnect  ws_send(b"0"+ring)  term.py:679    reader leaves  emit() fails
   grace deadline  ,              stop.set()  the turn DIES
```

This part settles that one build, the ring, closes three of the rows above, and that the other two were already delivered next door. The target was not a new architecture. It was the one QD3 already shipped. A terminal survives a reload because its bytes go to a RING that clients attach to. Chat's bytes went straight down the socket of whoever happened to ask. That single difference was the mechanism under three of the five rows above, and it is why the VS Code panel felt different rather than only nicer. - R1 · the ring, ported from `term.py` · [x] BUILT 260801, proven in a real browser 260802 `live/turnring.py` is the module. It holds one `Turn` per question key, and every event carries a counter `n` that only goes up. A 1MB/20k cap trims from the front and reports a `gap` instead of a quietly short stream. A turn also keeps a 600s grace window. `emit()` now pushes into it and `chat()` starts a runner thread, so the request is no longer the turn's owner but its FIRST READER. `POST /_board/attach {file, cursor}` is how a second reader joins. Proven by `checks/ring_e2e.py`, which is shaped like the complaint rather than like the code. Start a turn, hang up the socket the way navigating does, re-attach at the cursor, and demand the rest. It gets it: 117 events and an 11k-character answer that the first reader never saw. Two defects the wire test could not see and a real browser did, and that is the rule working. A FINISHED ring was re-attached on every 25s heartbeat, and it repainted its answer each time. Fixed: attach now declines a finished turn, because once a turn ends the transcript is the right source. And the cursor was keyed on `logKey()`, which folds in a session id that CHANGES mid-turn; fixed. The last client defect was attaching at a cursor near zero, so the chat box replayed instead of resuming. It closed 260802: `checks/guichat.mjs` T6 reloads mid-turn and rejoins at cursor 137, against a pre-reload cursor of 99 (States A9.1). `emit()` writes into a per-session outbox with a counter that only goes up, and it fans out to every attached client, instead of writing to one `wfile`. A reader who leaves stops being an event. There is no `stop.set()` at chat.py:628, and no `fut.cancel()` plus `host().evict()` in the `finally`. `gate()` keeps answering permission asks for the life of the TURN,

not for the life of the request. Closes: the turn dying on navigate, reload, config switch or timeout; the ten-minute turn; and . - R2 · the kept view · RETIRED 260801 to QD5, which had already built it The plan was to stop the chat box being rebuilt per page by making it a kept view attached to the ring by cursor, VS Code's webview answer to " ". Reading QD5's shell rather than its summary closed the row. The chat pane's src IS <page>.html?pane=chat, so the chat box is ALREADY its own document, and no page navigation can reach it. It is the same outcome one layer up, and a better one, because it removes the rebuild instead of recovering from it. What survives here is only the half QD5 does not touch. A replayed session still paints in a poorer form than a live one, and that stays in §8's second row as its own item. - R3 · dock at every width · RETIRED 260801 to QD5, same reading shell.py's own stylesheet already does it. @media(max-width:820px) switches the grid to grid-template-rows:1fr 5px 1fr and hides only the page list. So page and chat are both visible on a phone. The fork this page carried, overlay-with-a-way-back against collapse-to-a-tab, is answered by neither: the shell simply stacks them. So §9 is ONE build, not three, and the two that left were not dropped but already delivered next door. R1 stands alone because the turn dying with its HTTP response is a SERVER fact, three sites in live/chat.py, and no arrangement of panes reaches it. The blocker is named and it is not technical. QD1's HOLD keys on the SCOPE path, while the Law's stated reason is the shared jsonl. A channel a reader can rejoin changes what "one live window" means. So two named sessions on one page cannot be live together until that Law is re-ruled. CORRECTION, 260801, mine to make: this page briefly warned that R1's held /_board/attach would compete with QD5's /_events for the browser's six connections per origin. That was written from QD5's prose and is wrong. /_events holds nothing. It is one small JSON ask every 400 ms on a pooled connection. shell.py's own docstring explains that a held stream was built FIRST and then removed, exactly because it cost a connection. The budget question is still real, but smaller and shaped differently: the held connections are the terminal's WebSocket and R1's own two, the turn stream and an attach.

10 The ten things a reader does, and the check that holds each one

Two doors and ten gestures: how to open the chat, and what a reader does once inside.

```
TWO DOORS TO THE SAME CHAT
  .../QD2-chat-sdk.html?split   > three panes . strip: >_ TUI Chat .  GUI Chat
  .../QD2-chat-sdk.html         > the one-page board .  bottom-right . pick GUI

THE TEN THINGS A READER DOES                                  proven by
  (1) open it                    the strip, or the  button   T1 . T1b . T13
  (2) ask                         the answer is drawn as md     T3
  (3) read back while it works    scrolling up is respected     T5
  (4) stop it                     , one honest line            T12
  (5) leave mid-turn              it keeps running, you rejoin  T6
  (6) move to another page        the chat stays on its own     T10
  (7) change session              , or  New session           T11
  (8) change model or tier         Settings, remembered        T16
  (9) use it on a phone           it fits,  Page gets you back T14
  (10) hand it to the TUI          >_ TUI, and  GUI takes back T17
```

This part settles that every gesture a reader makes has a named check behind it, so none of them can quietly stop working. Every row above is a gesture, not a feature, and each one is held by an assertion in `checks/guichat.mjs`. Read this part as the manual and the test list at once. If a row here is wrong, the check that names it is the thing to run. **Opening it.** A board url with `?split` gives three panes, and the strip at the top carries `>_ TUI Chat` and `GUI Chat`. A plain board url gives the original one-page board, where the chat is the button at the bottom right, and that button asks which of the two you want. Being in the split is remembered, so a plain url after using the split gives you the split back. `?plain` is how you ask for the single page on purpose. **Asking, and reading while it works.** The answer is drawn as markdown while it streams, and tool calls appear as their own cards. Scrolling up during a turn is respected, and the next token will not drag you back down. Press to stop, and it says one thing about stopping rather than two. **Leaving, and coming back.** This is the part that used to lose work and no longer does. A turn does not belong to the browser window that started it. Navigate away, reload, or lock your phone, and it keeps running on the server. When you come back, the chat box rejoins the turn already in progress and the answer lands. Moving the page pane to another page does not touch the chat pane at all. **Sessions.** One question can hold many conversations. `Sessions` lists them, `New session` starts a fresh one, and picking an old one shows its real history read back from disk. Switching away and back returns the same messages in the same order. **The two chats are one question.** `QD1`'s Law is one live window per question, so the GUI and the TUI hand the same session back and forth instead of running side by side. The strip's two buttons are how you do it, and the handover keeps the transcript.